Sapienza Università di Roma
MSc in Engineering in Computer Science
and Artificial Intelligence
Laboratory of Advanced Programming 2025/2026
Hackathon exam - 5 March 2026

1 Mission briefing: "Welcome to Mars. Please Don't Die."

We are in **2036**. After a 12-hour shift of questionable architectural decisions at **SpaceY**, you are "promoted" to *Mars Operations* by being accidentally shipped to Mars while sleeping at your desk.

You wake up in a fragile habitat whose automation stack is partially destroyed. Devices speak incompatible dialects. Some of them stream telemetry. Others respond only to polling. Actuators are still reachable — if you can invoke them correctly.

Your mission: rebuild a **distributed automation platform** capable of ingesting heterogeneous sensor data, normalizing it into a unified internal representation, evaluating simple automation rules, and providing a real-time dashboard for habitat monitoring.

Failure means thermodynamic consequences.

2 General information

- **Duration:** 5 days
- **Team size:** 2 to 5 students
- **Submission deadline:** 10 March 2026, 23:59
- **Final discussion:** In-person presentation + demo (~20 minutes total per group). All members must attend.

Suggested internal milestones:

- Day 1: short pitch + user stories draft
- Day 2: architecture defined and event schema defined
- Day 4: full end-to-end system running via docker compose
- Day 5: documentation and slides finalized

3 What you are provided with

You will receive a **Docker container** simulating a heterogeneous IoT environment. You **must not modify** the provided container.

The simulator includes:

1. REST-based devices, which must be polled

2. Publish-based devices, emitting telemetry asynchronously to topics

3.1 How to run the simulator

To start the simulator, first load the image from the provided tar archive and then run the container by publishing port 8080:

```
docker load -i mars-iot-simulator-oci.tar
docker run --rm -p 8080:8080 mars-iot-simulator:multiarch_v1
```

All endpoints are served from the base URL `http://localhost:8080`. The OpenAPI documentation is available at `http://localhost:8080/docs` and `http://localhost:8080/openapi.json`. You can also verify the simulator and list available devices using:

```
curl -s http://localhost:8080/health
curl -s http://localhost:8080/api/discovery | jq
```

3.2 REST sensors

You can list the available REST sensors by calling the sensors endpoint. The response includes the sensor identifiers used throughout the system:

```
curl -s http://localhost:8080/api/sensors | jq
```

The table below summarizes how each REST sensor maps to its schema family:

REST sensor	**Schema**
greenhouse_temperature	rest.scalar.v1
entrance_humidity	rest.scalar.v1
co2_hall	rest.scalar.v1
hydroponic_ph	rest.chemistry.v1
water_tank_level	rest.level.v1
corridor_pressure	rest.scalar.v1
air_quality_pm25	rest.particulate.v1
air_quality_voc	rest.chemistry.v1

Exact definitions are provided in `SCHEMA_CONTRACT.md` and via OpenAPI.

3.3 Telemetry (stream)

Telemetry topics are discoverable via the topics endpoint:

```
curl -s http://localhost:8080/api/telemetry/topics | jq
```

Streams can be consumed either via SSE (server sent events) or WebSocket. Use the following endpoints to subscribe to a topic:

```
GET /api/telemetry/stream/{topic}
WS /api/telemetry/ws?topic={topic}
```

The following table maps each topic to its schema family:

Topic	**Schema**
mars/telemetry/solar_array	topic.power.v1
mars/telemetry/radiation	topic.environment.v1
mars/telemetry/life_support	topic.environment.v1
mars/telemetry/thermal_loop	topic.thermal_loop.v1
mars/telemetry/power_bus	topic.power.v1
mars/telemetry/power_consumption	topic.power.v1
mars/telemetry/airlock	topic.airlock.v1

Default publish interval: 5 seconds.

3.4 Actuator APIs

Actuator state can be inspected via the actuators endpoint:

```
curl -s http://localhost:8080/api/actuators | jq
```

To change a state, POST a JSON payload to the actuator resource, for example:

```
curl -X POST http://localhost:8080/api/actuators/cooling_fan \
  -H 'Content-Type: application/json' \
  -d '{"state":"ON"}'
```

Available actuators include:

- `cooling_fan`
- `entrance_humidifier`
- `hall_ventilation`
- `habitat_heater`

Actuators are controlled exclusively via REST.

4 What you must deliver

You must design and implement a distributed automation platform that:

1. Collects data from simulated devices (polling and/or stream depending on team size)
2. Normalizes heterogeneous payloads into a **standard internal event format**. You need to define and document such a standard, and provide a file named `input.md` as support for your solution.
3. Uses an **event-driven architecture** internally (message broker required)
4. Evaluates **simple event-triggered automation rules**
5. Maintains and exposes the **latest state** of each sensor (in-memory caching)
6. Provides a real-time dashboard. You must provide the user stories in `input.md`.

4.1 Persistence

- Historical sensor data persistence is **not** required.
- The latest state per sensor may be cached in memory.
- Automation rules **must** be persisted, so that they survive service restarts.

Rule persistence can be implemented using any kind of relational database (e.g., Postgres), noSQL database (e.g., MongoDB), or embedded database (e.g., SQLite). Complex schema design is not required; a simple rule table/schema is sufficient.

4.2 Authentication

Authentication is **not required**. There are no user accounts. The platform may be treated as single-tenant. All sensors are globally visible: there is no concept of ownership or association.

5 Functional requirements

5.1 Device ingestion and normalization

The system must:

- Ingest device data
- Convert incoming data into a unified internal event schema

5.2 Automation engine

The automation engine must support **simple if–then rules**.

```
IF <sensor_name> <operator> <value> [unit]
THEN set <actuator_name> to ON | OFF
```

Supported operators: `<, <=, =, >, >=`.
Example: `IF greenhouse_temperature > 28 °C THEN set cooling_fan to ON`

Rules must:

- Be evaluated dynamically on event arrival
- Be persisted
- Trigger an actuator state update when condition is met

5.3 Frontend dashboard

The frontend must provide a real-time monitoring dashboard. A widget-based layout *may* be used, but it is not mandatory: static dashboards are also acceptable.

Optional widget approach

- Widgets can be configurable (select sensor binding).
- Example widget types include live value, line chart (updates while the page is open), actuator toggle, or custom visualizations by data type.

The frontend must also provide a rule management interface and real-time updates via WebSocket or SSE.

6 Architectural constraints

Mandatory

- Use at least one message broker, e.g., ActiveMQ, Kafka, etc.
- Use multiple backend services (avoid a single monolithic service)
- Separate ingestion, processing, and presentation
- Persist automation rules
- Provide architecture diagrams

Strongly discouraged: Tight coupling between services

7 Technical requirements

7.1 Tech stack

- Backend: any language/framework
- Messaging: ActiveMQ, Kafka, equivalent
- Database: any relational, noSQL or embedded
- Frontend: any framework/library

7.2 Containerization

You must provide:

- A Dockerfile per service
- A single `docker-compose.yml` that starts backend services, message broker, frontend and simulator container.

The system must start with `docker compose up`. No manual setup after startup.

7.3 Baseline

All groups are expected to meet the following baseline requirements:

1. Event-driven architecture
2. Meaningful broker usage
3. Unified event schema
4. In-memory latest sensor state
5. Persistent rule storage
6. Real-time dashboard
7. Docker reproducibility

7.4 Tasks by group size

Groups of 5 are expected to deliver the full system and implement about 25 user stories.
Groups of 4 are expected to deliver the full system and implement about 20 user stories.
Groups of 3 are expected to deliver the full system and implement about 15 user stories.
Groups of 2 may omit telemetry stream handling (REST polling only) and implement about 10 user stories.

Note: user stories must be documented in `input.md`.

8 Deliverables

Submit a public GitHub repository containing:

- Full source code
- Dockerfiles, `docker-compose.yml`
- `input.md`
- Student_doc.md
- Architecture diagrams
- User stories + LoFi mockups
- Presentation slides

The repository must have the following structure:

```
.
├── input.md
├── Student_doc.md
├── source/
└── booklets/
```

All the textual documents must follow the Markdown syntax. Please refer to `https://www.markdownguide.org/basic-syntax/` if you are not entirely familiar with it.

Following is the list of textual files to be submitted:

- `input.md` describes the system overview, the user stories, the standard event schema and the rule model. A (partial) example is accessible at `https://drive.google.com/file/d/1klvQ8r8WKfW86HS8HQ801QZh-yXZO5sK/view?usp=sharing`.
- `Student_doc.md`, which contains the specifics of the deployed system. Structure of the file: `https://drive.google.com/file/d/1stCQoen6ojT3hBexAkyp0Ja8H6XzOuFn/view?usp=sharing`. An example is accessible at `https://drive.google.com/file/d/15lmOqwYTG4qORk3bZgwMlRBWLaA5Gn-I/view?usp=sharing`.

The `booklets` folder contains whatever document you consider appropriate (slides, images, etc.).

The `source` folder contains the system developed in whatever technologies/framework you may want. In this folder all the source code, configuration files, and any other file you may need (please remember we have adopted an IaC approach - Infrastructure as Code) AND the Dockerfiles, docker-compose files, etc. which will allow the instructors to re-build/re-deploy your system on whatever platform (either on-premise or on cloud).

Each group needs to create a repo named `<MATRICOLA>_<PROJECT>` where `<MATRICOLA>` is the INFOSTUD Student ID of the group leader and `<PROJECT>` is the acronym of your project/group.

In order to avoid misunderstandings, this is repeated. Before the discussion, you need to share with the instructor all the documents and materials of your project, in particular the requirements (as user stories) of your entire system. User stories can be documented by using a spreadsheet and collecting all of them in a booklet. For each user story, it should be provided also a LoFi mockup, to be prepared with whatever tool (e.g., Balsamiq, Figma, etc.), and a textual description highlighting specific non functional requirements (if any).

The system is developed in whatever technologies/framework you may want. The release should be done by providing the link of a GitHub repo with all the source code, configuration files, any other file you may need (please remember we have adopted a IaC approach - Infrastructure as Code) AND the Dockerfiles, docker-compose files, etc. which will allow the instructors to re-build/re-deploy your system on whatever platform (either on-premise or on cloud).